CEG 2136: Computer Architecture I

Laboratory 3: Design and Simulation of ALU

1 Objective

This laboratory requires students to design and simulate the execution unit (datapath) of a Central Processing Unit (CPU). Students will complete the following tasks:

- Design a 4-bit execution unit comprised of an arithmetic and logic circuit.
- Implement 16 distinct microoperations that process two 4-bit operands.
- Derive and implement status register logic to track overflow (V), zero (Z), negative (N), and carry (Cy) conditions.
- Simulate the complete datapath using a provided input generator to supply the operands (A, B) and control signals ($S_3 \ldots S_0$).
- Download and run the design on the Altera DE2-115 board to experimentally verify the circuit.

Textbook: *Computer System Architecture*, Morris Mano, 3rd Ed.
Recommended Reading: Section 2.4 (Registers), Section 3.3 (Fixed-Point Representation), and Chapter 4 (Register Transfer and Microoperations)

2 Introduction

A Central Processing Unit (CPU) consists of two primary components: a control unit (CU) and an execution unit (EU), also known as the datapath. In this laboratory, students design the datapath to process two 4-bit operands and generate a 4-bit result.

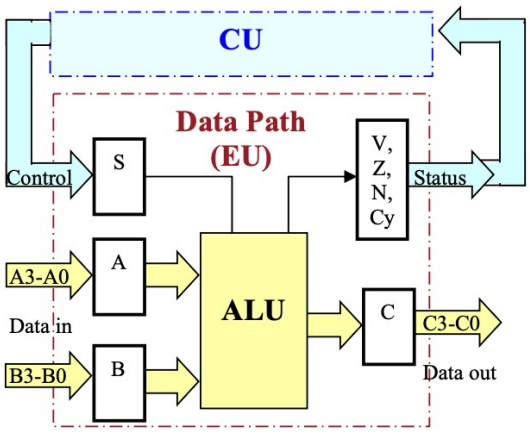


Figure 1: CPU block diagram - Control Unit (CU) and Execution Unit (EU).

2.1 Functional Specification

The datapath stores data in buffer registers (D flip-flops). Register A and Register B hold the incoming operands, while Register C stores the final output. The system is synchronous; because all D flip-flops are driven by the same clock signal, the reading of inputs and the writing of outputs occur within the same clock cycle.

The Arithmetic Logic Unit (ALU) operates as a combinational circuit capable of executing 16 distinct operations. A 4-bit control signal ($S_3 \dots S_0$) dictates the operation:

- **Mode Select (S_3):** Routes the operands to either the Arithmetic Circuit (AC) or the Logic and Shift Circuit (LSC).
- **Operation Select ($S_2 \dots S_0$):** Defines the specific arithmetic or logical operation performed within the selected circuit.

The system evaluates the ALU output to generate four status flags: Overflow (V), Zero (Z), Negative (N), and Carry (Cy).

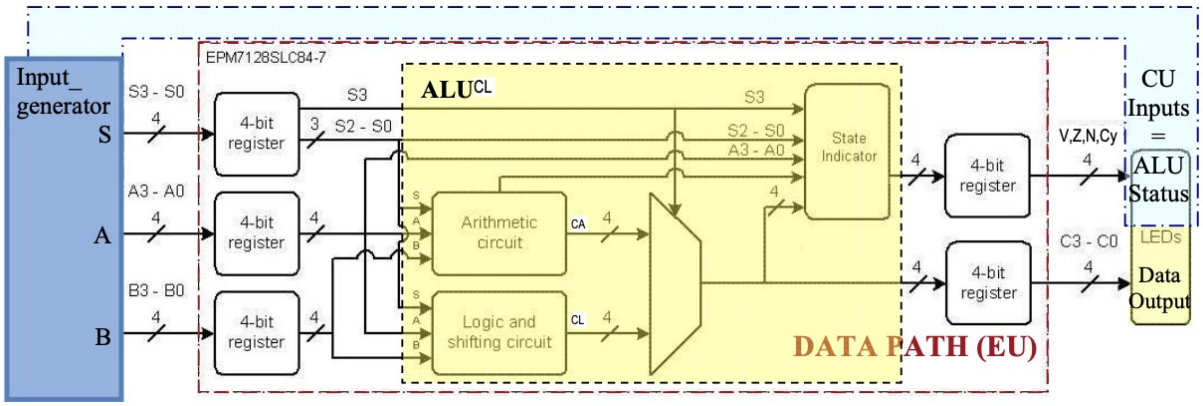


Figure 2: Block diagram of the datapath with `input_generator`.

To facilitate testing and simulation, the design incorporates a pre-designed component named `input_generator_high_freq`. This module automatically generates the sequence of test vectors for the operands (A, B) and the control signals (S), as illustrated above in Figure 2.

2.2 Relevance to Computer Architecture

The datapath is the core execution unit of a processor, responsible for performing micro-operations on register data [3]. Designing the arithmetic and logic circuits as distinct, modular blocks reduces hardware complexity. Selector signals then route the appropriate result to the final output. Finally, status bits (V, Z, N, Cy) provide immediate feedback to the control unit, enabling the processor to evaluate data conditions (e.g., arithmetic overflow) and adjust its control flow accordingly [3].

3 Pre-Lab

Complete the following designs before beginning the software implementation.

Table 1: Arithmetic Operations

Select				Operation	Description
S_3	S_2	S_1	S_0		
0	0	0	0	$CA_i = A + B$	Add
0	0	0	1	$CA_i = A + B + 1$	Add with carry
0	0	1	0	$CA_i = A$	Transfer A
0	0	1	1	$CA_i = A + 1$	Increment A
0	1	0	0	$CA_i = A + \overline{B}$	Subtract with borrow (1's complement)
0	1	0	1	$CA_i = A + \overline{B} + 1$	Subtract (2's complement)
0	1	1	0	$CA_i = \overline{A}$	Complement of A
0	1	1	1	$CA_i = \overline{A} + 1$	2's Complement of A

Table 2: Logic and Shift Operations

Select				Operation	Description
S_3	S_2	S_1	S_0		
1	0	0	0	$CL_i = 0$	Reset
1	0	0	1	$CL_i = 1$	Set
1	0	1	0	$CL_i = A \wedge B$	A AND B
1	0	1	1	$CL_i = A \vee B$	A OR B
1	1	0	0	$CL_i = A \oplus B$	A XOR B
1	1	0	1	$CL_i = A \wedge \overline{B}$	Reset A bits selected by *mask* B
1	1	1	0	$CL_i =$ ashl A (A_{i-1})	Arithmetic Shift Left (signed multiplication by 2)
1	1	1	1	$CL_i =$ ashr A (A_{i+1})	Arithmetic Shift Right (signed division by 2)

3.1 Design of the Logic and Shift Circuit (LSC)

1. Implement the missing logic gates required to complete the 1-bit logic diagram (LSC_i) shown in Figure 3. Base your design on the logic and shift operations defined in Table 2.

2. Draw the block diagram for the complete 4-bit LSC.

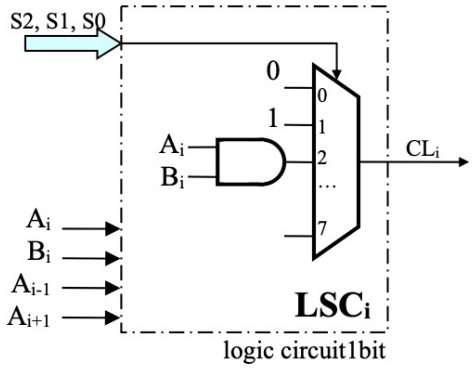


Figure 3: Incomplete 1-bit logic and shift circuit.

3.2 Design of the Arithmetic Circuit (AC)

1. Complete the truth table for the arithmetic circuit (Table 3) based on the operations defined in Table 1.

2. Design a logic solution utilizing a 4-to-1 Multiplexer (MUX) to generate the intermediate outputs op_0 and op_1. Use S_2 and S_1 as the MUX selector lines.

3. Derive the Boolean equation for the carry signal (Cy) in terms of the select lines (S_2, S_1, S_0).

4. Draw the block diagram of the 1-bit arithmetic circuit.

5. Draw the block diagram of the full 4-bit arithmetic circuit by cascading four of your 1-bit circuits.

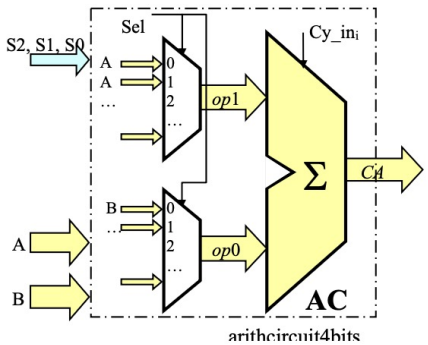


Figure 4: Incomplete 4-bit arithmetic circuit. Note that Σ is based on a full adder.

Table 3: Arithmetic Circuit Truth Table (Incomplete)

Select			**MUX Output**		**Carry**	**Output**
S_2	S_1	S_0	op_0	op_1	Cy_{in}	CA
0	0	0	A	B	0	$CA = A + B$ (Add)
0	0	1	A	B	1	$CA = A + B + 1$ (Add with carry)
0	1	0	A	0	0	$CA = A$ (Transfer A)
0	1	1				$CA = A + 1$
1	0	0				$CA = A + \overline{B}$
1	0	1				$CA = A + \overline{B} + 1$
1	1	0				$CA = \overline{A}$
1	1	1				$CA = \overline{A} + 1$

3.3 ALU Status Register Equations

Derive the logic equations for the four status bits. Implement them using conventional logic gates based on the following functional definitions:

- **Cy (Carry):** Set to 1 only when the operation is an arithmetic operation and its carry output is 1.
- **N (Negative):** Set to 1 if the most significant bit of the result (C_3) is 1.
- **Z (Zero):** Set to 1 only if the output of the ALU contains all 0's.

- **V (Overflow):** Set to 1 when an overflow occurs during 2's complement operations. For this lab, implementing overflow detection for signed arithmetic (addition/subtraction) is sufficient. Handling overflow for shift operations (e.g., arithmetic shift left) is optional.

4 Laboratory

4.1 Using Bus Connections

To minimize routing errors and reduce visual clutter, students are strongly encouraged to use bus connections for all 4-bit data paths (e.g., `A[3..0]`, `B[3..0]`). Please refer to the Appendix A.4 for more detailed instructions.

4.2 Project File Hierarchy

This laboratory requires more signal routing than prior labs. To reduce troubleshooting complexity, students should take a bottom-up approach by building and verifying the foundational sub-modules first. Figure 5 illustrates the recommended file hierarchy for this process. Before starting your design, review Section 4.6 to see the available multiplexers and Figure 7 for a complete top-level reference diagram..

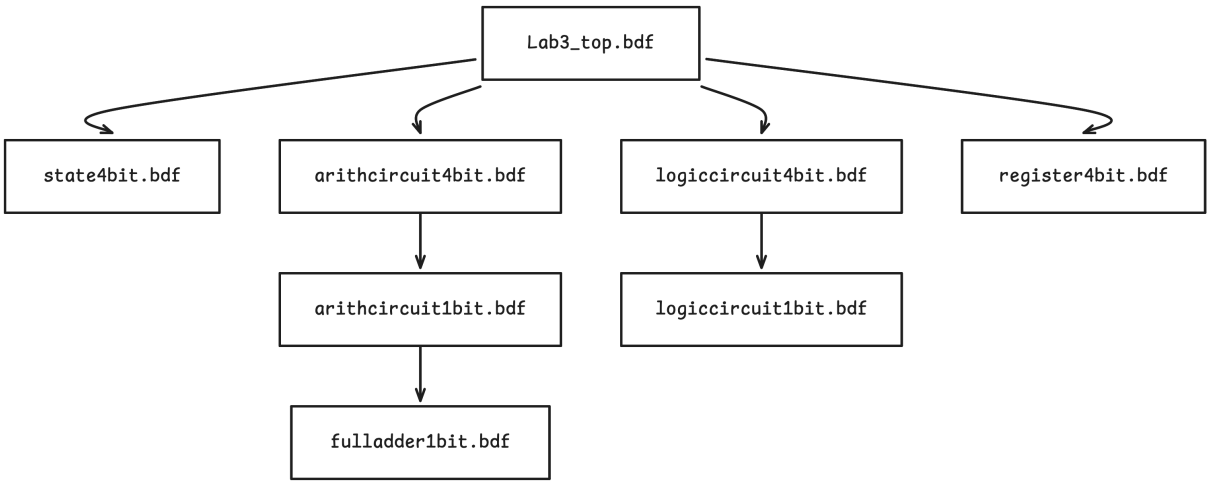


Figure 5: Recommended project file hierarchy.

4.3 Project Setup

1. Launch **Quartus II** and create a new project.
2. Save the project in your `H:` network drive.
3. Configure the target device as follows:
   - Device Family: Cyclone IV E
   - Device: EP4CE115F29C7
4. Ensure the project name and top-level entity are set appropriately.

4.4 Schematic Design: Sub-Modules

1. Design the 1-bit full adder (`fulladder1bit.bdf`) and convert it into a symbol.
2. Design the 1-bit arithmetic circuit (`arithcircuit1bit.bdf`) based on your pre-lab, utilizing your full adder symbol. Convert it into a symbol file.
3. Design the 4-bit arithmetic circuit (`arithcircuit4bit.bdf`) by instantiating four 1-bit arithmetic symbols. Convert it into a symbol.
4. Design the 1-bit logic circuit (`logiccircuit1bit.bdf`) based on your pre-lab. Convert it into a symbol.
5. Design the 4-bit logic circuit (`logiccircuit4bit.bdf`) by instantiating four 1-bit logic symbols. Convert it into a symbol.
6. Design the 4-bit D-register (`register4bit.bdf`). Convert it into a symbol.
7. Implement the status equations (`state4bit.bdf`). Convert it into a symbol.

4.5 Schematic Design: Top-Level Entity

1. Create the top-level schematic (`Lab3_top.bdf`).
2. Instantiate the `register4bit` blocks for registers A, B, and C.
3. Instantiate the arithmetic, logic, and status blocks.

4. Integrate the provided high frequency input generator into your schematic, as shown in Figure 6. The input generator automatically feeds operands and control signals into the datapath for simulation.

5. Route all sub-modules together, referring to Figure 7.

6. Compile the project and verify it completes with zero errors.

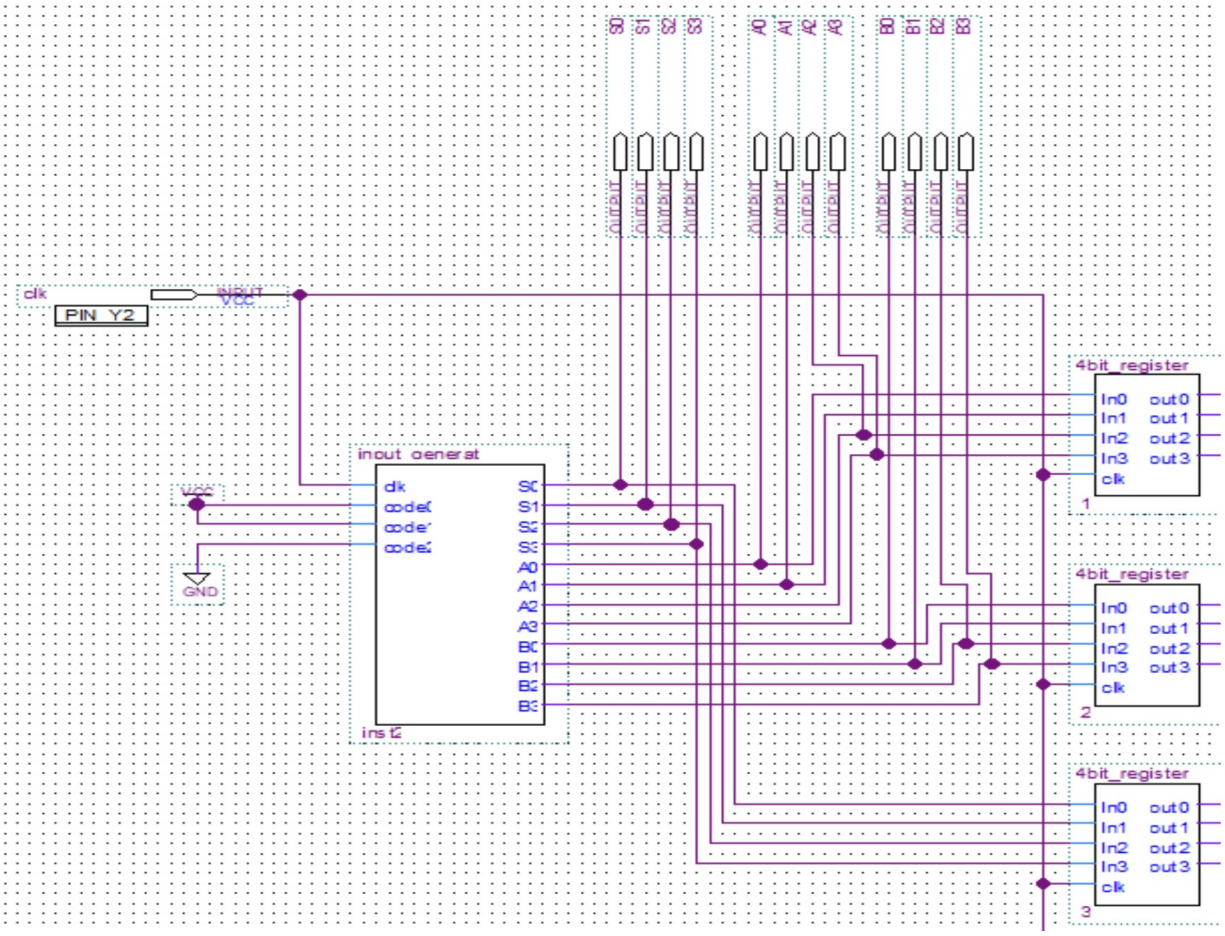


Figure 6: Schematic illustrating the connection of the input generator to the 4-bit registers. Ensure that the `code2`, `code1`, and `code0` inputs are set to 0, 1, and 1, respectively.

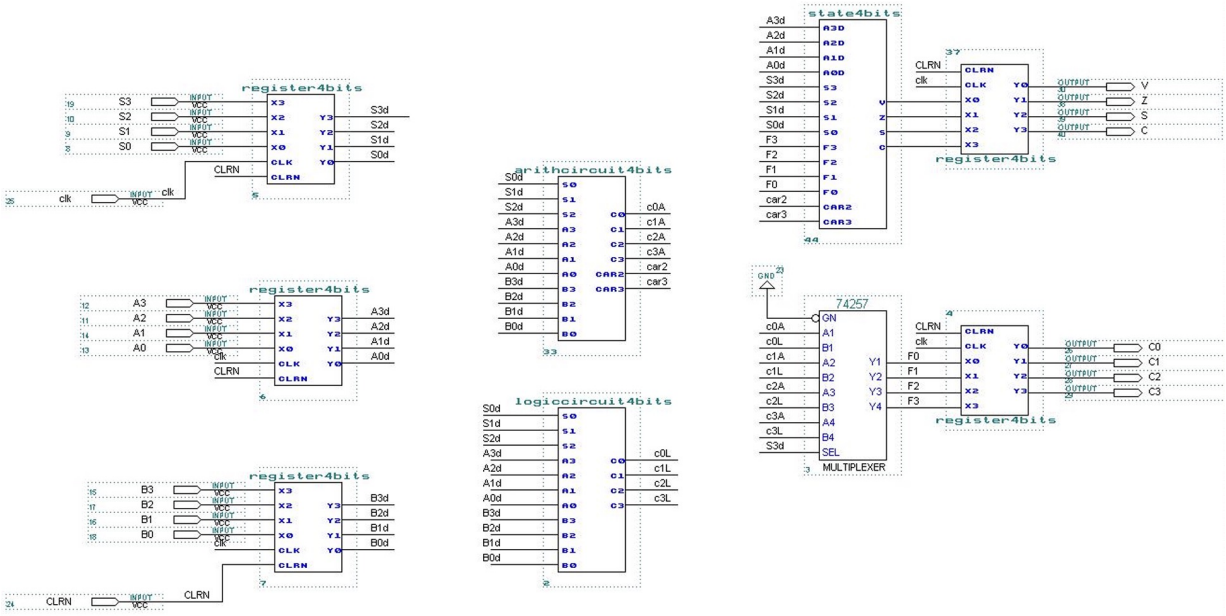
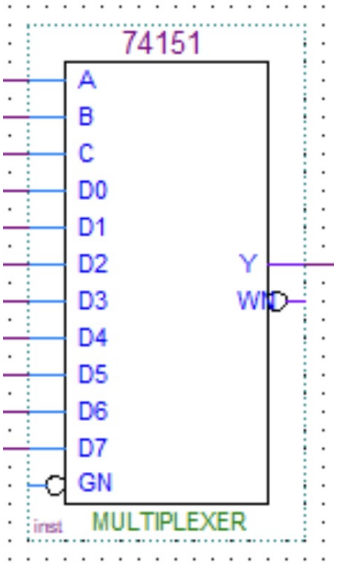


Figure 7: Top-level block diagram of the complete ALU.

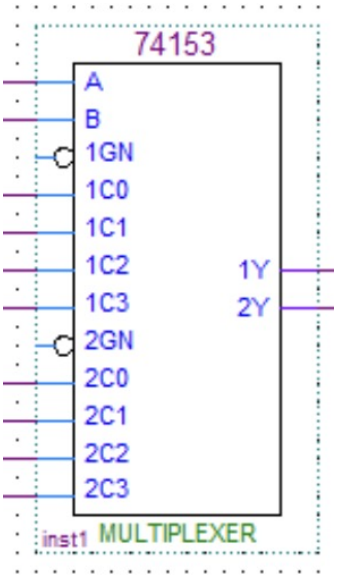
4.6 Available Multiplexers

In the following are given the functional descriptions (symbol and truth table) of some of the multiplexers you may want to use to implement your ALU circuit.



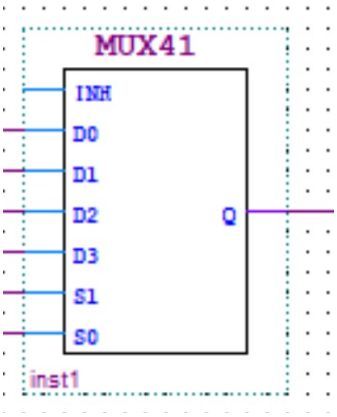
Select			Enable	Output
C	B	A	GN	Y
X	X	X	1	0
0	0	0	0	D0
0	0	1	0	D1
0	1	0	0	D2
0	1	1	0	D3
1	0	0	0	D4
1	0	1	0	D5
1	1	0	0	D6
1	1	1	0	D7

Figure 8: Symbol and truth table for the `74151` 8-to-1 multiplexer.



Select		Enable	Output
B	A	GN	Y
X	X	1	0
0	0	0	C0
0	1	0	C1
1	0	0	C2
1	1	0	C3

Figure 9: Symbol and truth table for the `74153` dual 4-to-1 multiplexer.



Select		Enable	Output
S_1	S_0	INH	Q
X	X	1	0
0	0	0	D0
0	1	0	D1
1	0	0	D2
1	1	0	D3

Figure 10: Symbol and truth table for the `mux41` 4-to-1 multiplexer.

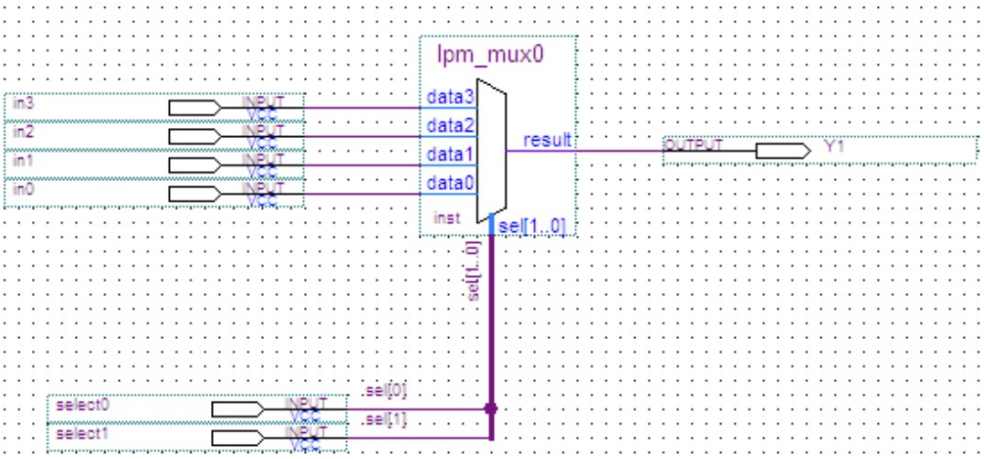


Figure 11: Example of a multiplexer lpm-mux

Note: You are also free to use the parameterized `lpm_mux` component if you find it simpler to implement or are already familiar with it from previous designs.

5 Simulation and Verification

1. Create a new University Program waveform file (VWF).

2. Add the clock signal, input buses (A, B, S), output bus (C), and status bits (V, Z, N, Cy) to the waveform viewer using the **Node Finder**.

   - If you used bus connections in your top-level schematic, the 4-bit signals will automatically appear as grouped buses in the waveform viewer.
   - Otherwise, you should manually group these bits together to simplify your analysis. Refer to Appendix A.6 for detailed software instructions.

3. Set the radix of the buses to **Hexadecimal** to format the values as A_{16}, B_{16}, and C_{16}. You should also group the four status registers together to form a single hexadecimal status word (St_{16}).

4. Assign a clock signal with a 5 ns period to the clock input. The `input_generator_high_freq` module will automatically supply the values for A, B, and S.

5. Verify that the **Quartus II Simulator** is selected in the simulation settings.

6. Run a functional simulation.

7. Evaluate the output waveforms by observing the result bus (C) and the ALU state indicators (V, Z, N, Cy) for each microoperation and verify that the observed behavior matches the waveform shown in Figure 12.

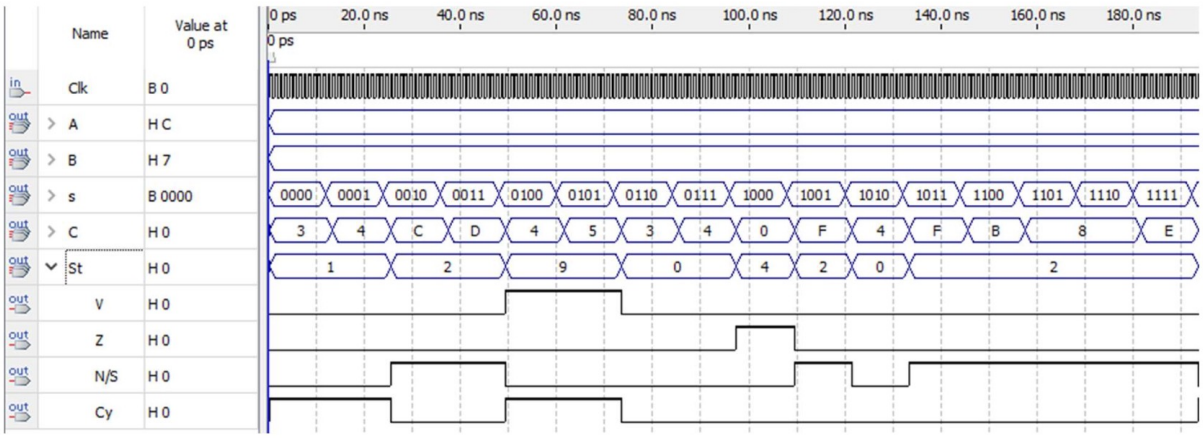


Figure 12: Expected simulation output for verification

6 Demonstration

This section allows students to verify the design using the DE2-115 board. The clock signal (CLK) is generated from a debounced push-button, and the result and status flags are displayed on the LEDs for observation.

6.1 Pin Assignment

1. Select **Assignments** $>$ **Assignment Editor**.
2. Use the **Node Finder** to import the clock input (CLK), the output bus ($C[3:0]$), and the status bits (V, Z, N, Cy) into the editor.
3. Assign physical locations to each signal according to Table 4.
4. Save the assignments and recompile the project.

Table 4: Pin Assignments

Signal	**Component**	**Pin Location Value**
$Clock$	KEY0	PIN_M23
C_3	LEDR3	PIN_F21
C_2	LEDR2	PIN_E19
C_1	LEDR1	PIN_F19
C_0	LEDR0	PIN_G19
V (Overflow)	LEDG3	PIN_E24
Z (Zero)	LEDG2	PIN_E25
N (Negative)	LEDG1	PIN_E22
Cy (Carry)	LEDG0	PIN_E21

6.2 FPGA Programming and Verification

1. Connect the USB-Blaster cable between the PC and the FPGA board.
2. Set the RUN/PROG switch (SW19, leftmost toggle switch) to **RUN**.

3. Open the programmer: select **Tools** > **Programmer**.
4. Verify that the USB-Blaster is selected in Hardware Setup.
    - If the USB-Blaster does not appear in the list of available hardware, close and reopen the window.
    - You may need to unplug and replug the cable or power-cycle the board.
5. Ensure your compiled `.sof` output file is loaded in the programmer.
6. Check the **Program/Configure** box and click **Start**.
7. Press the push-button (**KEY0**) to step the clock manually.
8. Verify that the observed sequence matches the behavior shown in the simulation waveform.
9. Demonstrate the working design to a Teaching Assistant for verification.

7 Lab Completion

There is no formal lab report or code submission required. Instead, before leaving the lab, you must:

- Demonstrate your functional design to the TA.
- Answer questions from the TA to assess your individual understanding of the lab concepts.
- Back up your project files to your University OneDrive.

A Appendix I: Best Practices and Troubleshooting

Use the following references and techniques to guide your laboratory work.

A.1 Project Organization

Always create dedicated folders for each Quartus project. Because laboratory computers erase local files when powered off, you must save and back up your project folders using your University OneDrive to prevent data loss.

A.2 Debugging Connections

To verify signal connectivity in the schematic editor, double-click a wire segment. Quartus highlights the entire connected net, as shown in Figure 13, allowing you to confirm that an input is connected to its intended destination.

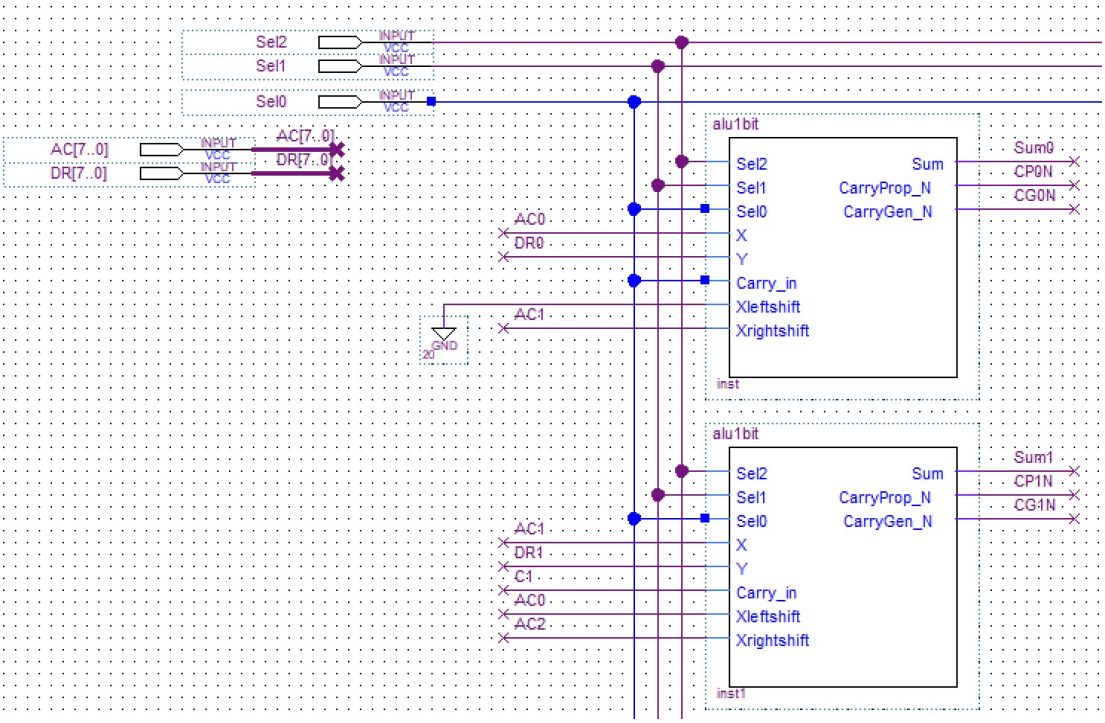


Figure 13: Quartus highlighting a connected net upon double-clicking a wire segment.

A.3 Naming Wires

Name wires to make Quartus recognize connections without drawing the entire physical path [1], as shown in Figure 14.

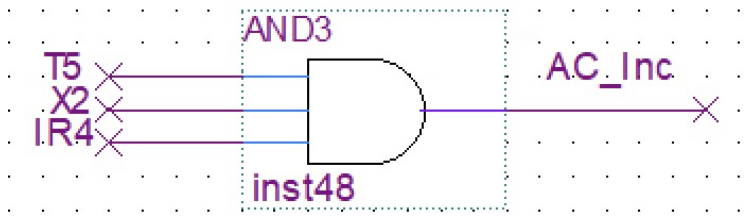


Figure 14: Named wires create logical connections without physical lines.

A.4 Using Bus Connections

Replace multiple individual wires with bus connections to simplify complex schematic designs. Buses reduce visual clutter and lower the probability of wiring errors when routing multiple inputs and outputs. This practice becomes critical as circuit designs grow.

A bus carries multiple data bits. A bus of three signals utilizes the naming format `X[2..0]` (count down). In Quartus, buses appear as thick wires. You can extract individual signals from buses and use them as regular inputs [4], as shown in Figure 15.

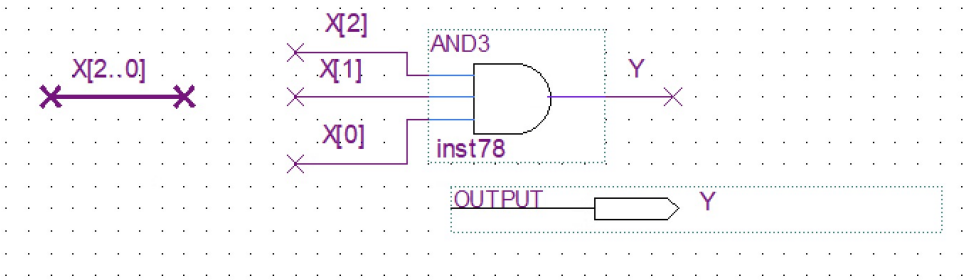


Figure 15: Bus in Quartus, X[2..0], with individual signals used as regular inputs.

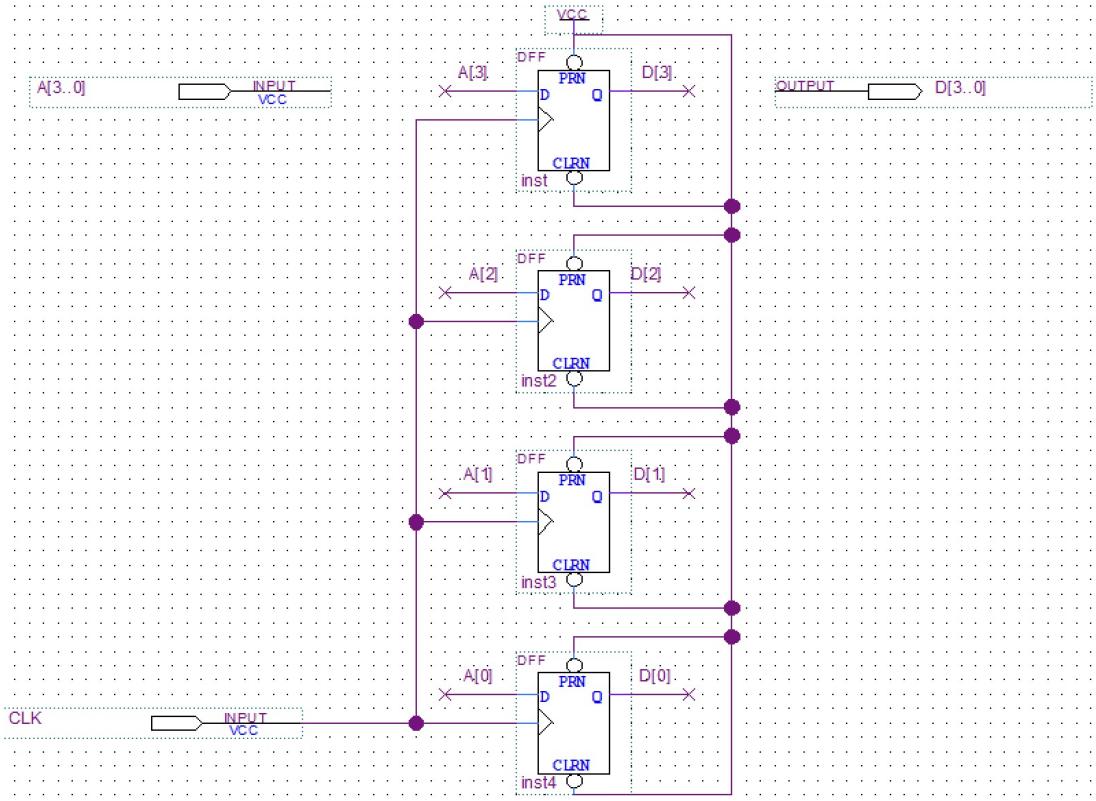


Figure 16: Flip-flop alternate design utilizing bus connections for inputs and outputs.

A.5 Using Wires

In Quartus schematic entry (.bdf) files, you can use a *wire* primitive to assign multiple names to the same signal [4], as illustrated in Figure 17. A *wire* primitive looks like an inverter without the bubble.

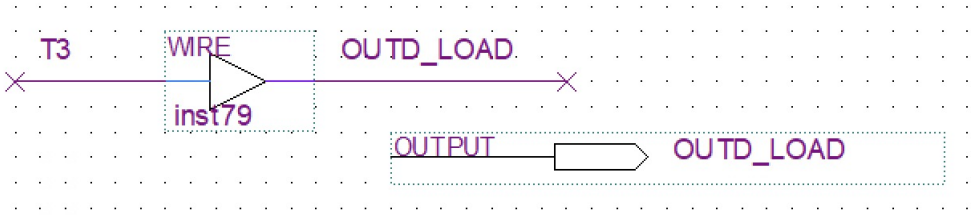


Figure 17: Wire primitive assigning multiple names to a single signal.

A.6 Simplifying Simulation Analysis

In the waveform editor, group related input or output signals together. Select multiple inputs, right-click, choose **Grouping**, and select **Group**.

Once grouped, the radix (base) of the group can be modified. Right-click the group, select **Radix**, and choose binary, hexadecimal, unsigned decimal, or octal. This allows you to input and read values as single numbers (e.g., A = 1011), which speeds up verification, as shown in Figure 18. When grouping signals, ensure that the bits are ordered from most significant bit (MSB) to least significant bit (LSB) [1].

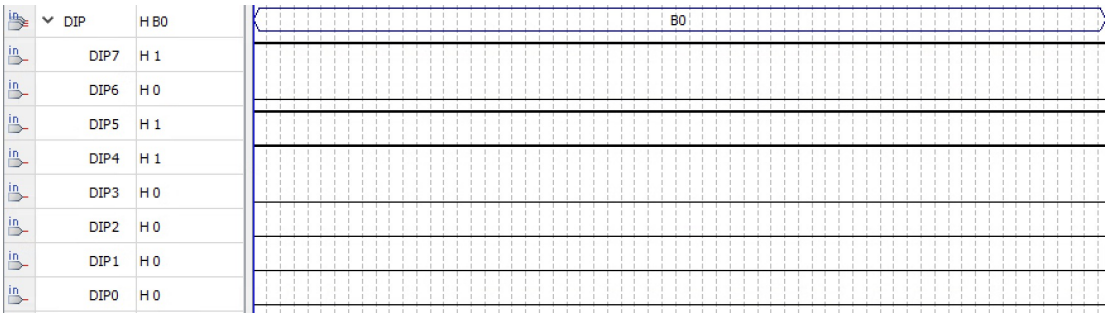


Figure 18: Grouped input signals utilizing a hexadecimal radix.

Note: If bus connections are used in the schematic, Quartus automatically groups these signals in the waveform editor.

A.7 Quartus Software Glitches (Simulation or Compilation)

Occasionally, Quartus workspace files can become corrupted. You might be experiencing this if:

- The simulation window opens and immediately closes without displaying any waveforms.
- The compilation process gets completely stuck, freezes, or runs unusually slowly.

Solution: Create a brand new Quartus project in a new folder. Copy over *only* your essential source files (such as your `.bdf`, `.bsf`, `.v`, `.vhd`, and `.vwf` files) into the new project and recompile.

A.8 Hardware Connection Issues (USB-Blaster)

When attempting to program the FPGA, the USB-Blaster might occasionally fail to appear in the list of available hardware. Try the following:

1. Close and reopen the Hardware Setup window in the Programmer.
2. Unplug and replug the USB cable connecting the board to the PC.
3. Power-cycle the DE2-115 board (turn the red power button off, wait a few seconds, and turn it back on).

Solution: If the USB-Blaster still fails to connect after trying all of the above steps, the specific test board, cable, or PC USB port may be faulty. In this case, try moving to another station and try again.

A.9 Additional Resources

Consult the provided course reference documents for detailed information on Altera Design workflows. The following resources may also be useful for supplementing the Virtual Campus material.

- Quartus Troubleshooting Guide (Colorado State University)
- Quartus Wire and Bus Connections (University of Florida)
- Quartus Bus Fundamentals (University of Florida)
- Memory Initialization File (MIF) Creation in Quartus (University of Florida)

B Appendix II: DE2-115 Board I/O Pin Assignments

This appendix provides the FPGA pin assignments for the Altera DE2-115 board, including switches, LEDs, buttons, and 7-segment displays [2].

B.1 Slide Switches and Push-buttons

Table 5: Pin Assignments for Slide Switches

Signal Name	**FPGA Pin No.**	**Description**
SW[0]	PIN_AB28	Slide Switch [0]
SW[1]	PIN_AC28	Slide Switch [1]
SW[2]	PIN_AC27	Slide Switch [2]
SW[3]	PIN_AD27	Slide Switch [3]
SW[4]	PIN_AB27	Slide Switch [4]
SW[5]	PIN_AC26	Slide Switch [5]
SW[6]	PIN_AD26	Slide Switch [6]
SW[7]	PIN_AB26	Slide Switch [7]
SW[8]	PIN_AC25	Slide Switch [8]
SW[9]	PIN_AB25	Slide Switch [9]
SW[10]	PIN_AC24	Slide Switch [10]
SW[11]	PIN_AB24	Slide Switch [11]
SW[12]	PIN_AB23	Slide Switch [12]
SW[13]	PIN_AA24	Slide Switch [13]
SW[14]	PIN_AA23	Slide Switch [14]
SW[15]	PIN_AA22	Slide Switch [15]
SW[16]	PIN_Y24	Slide Switch [16]
SW[17]	PIN_Y23	Slide Switch [17]

Table 6: Pin Assignments for Push-button (Debounced) Switches

Signal Name	**FPGA Pin No.**	**Description**
KEY[0]	PIN_M23	Push-button [0]
KEY[1]	PIN_M21	Push-button [1]
KEY[2]	PIN_N21	Push-button [2]
KEY[3]	PIN_R24	Push-button [3]

B.2 LED Assignments

Table 7: Pin Assignments for LEDs

Signal Name	FPGA Pin No.	Description
LEDR[0]	PIN_G19	LED Red [0]
LEDR[1]	PIN_F19	LED Red [1]
LEDR[2]	PIN_E19	LED Red [2]
LEDR[3]	PIN_F21	LED Red [3]
LEDR[4]	PIN_F18	LED Red [4]
LEDR[5]	PIN_E18	LED Red [5]
LEDR[6]	PIN_J19	LED Red [6]
LEDR[7]	PIN_H19	LED Red [7]
LEDR[8]	PIN_J17	LED Red [8
LEDR[9]	PIN_G17	LED Red [9]
LEDR[10]	PIN_J15	LED Red [10]
LEDR[11]	PIN_H16	LED Red [11]
LEDR[12]	PIN_J16	LED Red [12]
LEDR[13]	PIN_H17	LED Red [13]
LEDR[14]	PIN_F15	LED Red [14]
LEDR[15]	PIN_G15	LED Red [15]
LEDR[16]	PIN_G16	LED Red [16]
LEDR[17]	PIN_H15	LED Red [17]
LEDG[0]	PIN_E21	LED Green [0]
LEDG[1]	PIN_E22	LED Green [1]
LEDG[2]	PIN_E25	LED Green [2]
LEDG[3]	PIN_E24	LED Green [3]
LEDG[4]	PIN_H21	LED Green [4]
LEDG[5]	PIN_G20	LED Green [5]
LEDG[6]	PIN_G22	LED Green [6]
LEDG[7]	PIN_G21	LED Green [7]
LEDG[8]	PIN_F17	LED Green [8]

B.3 Clock Inputs

Table 8: Pin Assignments for Clock Inputs

Signal Name	**FPGA Pin No.**	**Description**
CLOCK_50	PIN_Y2	50 MHz clock input
CLOCK2_50	PIN_AG14	50 MHz clock input
CLOCK3_50	PIN_AG15	50 MHz clock input
SMA_CLKOUT	PIN_AE23	External (SMA) clock output
SMA_CLKIN	PIN_AH14	External (SMA) clock input

B.4 7-Segment Displays

Table 9: Pin Assignments for 7-Segment Displays (HEX0 to HEX3)

Signal Name	**FPGA Pin No.**	**Description**
HEX0[0]	PIN_G18	Seven Segment Digit 0[0]
HEX0[1]	PIN_F22	Seven Segment Digit 0[1]
HEX0[2]	PIN_E17	Seven Segment Digit 0[2]
HEX0[3]	PIN_L26	Seven Segment Digit 0[3]
HEX0[4]	PIN_L25	Seven Segment Digit 0[4]
HEX0[5]	PIN_J22	Seven Segment Digit 0[5]
HEX0[6]	PIN_H22	Seven Segment Digit 0[6]
HEX1[0]	PIN_M24	Seven Segment Digit 1[0]
HEX1[1]	PIN_Y22	Seven Segment Digit 1[1]
HEX1[2]	PIN_W21	Seven Segment Digit 1[2]
HEX1[3]	PIN_W22	Seven Segment Digit 1[3]
HEX1[4]	PIN_W25	Seven Segment Digit 1[4]
HEX1[5]	PIN_U23	Seven Segment Digit 1[5]
HEX1[6]	PIN_U24	Seven Segment Digit 1[6]
HEX2[0]	PIN_AA25	Seven Segment Digit 2[0]
HEX2[1]	PIN_AA26	Seven Segment Digit 2[1]
HEX2[2]	PIN_Y25	Seven Segment Digit 2[2]
HEX2[3]	PIN_W26	Seven Segment Digit 2[3]
HEX2[4]	PIN_Y26	Seven Segment Digit 2[4]
HEX2[5]	PIN_W27	Seven Segment Digit 2[5]
HEX2[6]	PIN_W28	Seven Segment Digit 2[6]
HEX3[0]	PIN_V21	Seven Segment Digit 3[0]
HEX3[1]	PIN_U21	Seven Segment Digit 3[1]
HEX3[2]	PIN_AB20	Seven Segment Digit 3[2]
HEX3[3]	PIN_AA21	Seven Segment Digit 3[3]
HEX3[4]	PIN_AD24	Seven Segment Digit 3[4]
HEX3[5]	PIN_AF23	Seven Segment Digit 3[5]
HEX3[6]	PIN_Y19	Seven Segment Digit 3[6]

Table 10: Pin Assignments for 7-Segment Displays (HEX4 to HEX7)

Signal Name	**FPGA Pin No.**	**Description**
HEX4[0]	PIN_AB19	Seven Segment Digit 4[0]
HEX4[1]	PIN_AA19	Seven Segment Digit 4[1]
HEX4[2]	PIN_AG21	Seven Segment Digit 4[2]
HEX4[3]	PIN_AH21	Seven Segment Digit 4[3]
HEX4[4]	PIN_AE19	Seven Segment Digit 4[4]
HEX4[5]	PIN_AF19	Seven Segment Digit 4[5]
HEX4[6]	PIN_AE18	Seven Segment Digit 4[6]
HEX5[0]	PIN_AD18	Seven Segment Digit 5[0]
HEX5[1]	PIN_AC18	Seven Segment Digit 5[1]
HEX5[2]	PIN_AB18	Seven Segment Digit 5[2]
HEX5[3]	PIN_AH19	Seven Segment Digit 5[3]
HEX5[4]	PIN_AG19	Seven Segment Digit 5[4]
HEX5[5]	PIN_AF18	Seven Segment Digit 5[5]
HEX5[6]	PIN_AH18	Seven Segment Digit 5[6]
HEX6[0]	PIN_AA17	Seven Segment Digit 6[0]
HEX6[1]	PIN_AB16	Seven Segment Digit 6[1]
HEX6[2]	PIN_AA16	Seven Segment Digit 6[2]
HEX6[3]	PIN_AB17	Seven Segment Digit 6[3]
HEX6[4]	PIN_AB15	Seven Segment Digit 6[4]
HEX6[5]	PIN_AA15	Seven Segment Digit 6[5]
HEX6[6]	PIN_AC17	Seven Segment Digit 6[6]
HEX7[0]	PIN_AD17	Seven Segment Digit 7[0]
HEX7[1]	PIN_AE17	Seven Segment Digit 7[1]
HEX7[2]	PIN_AG17	Seven Segment Digit 7[2]
HEX7[3]	PIN_AH17	Seven Segment Digit 7[3]
HEX7[4]	PIN_AF17	Seven Segment Digit 7[4]
HEX7[5]	PIN_AG18	Seven Segment Digit 7[5]
HEX7[6]	PIN_AA14	Seven Segment Digit 7[6]

References

[1] Colorado State University. Quartus troubleshooting guide. https://www.engr.colostate.edu/ECE102/Spring22/QUARTUS%20TROUBLESHOOTING%20GUIDE.pdf. Accessed: Feb. 2026.

[2] Iowa State University. Pin assignment tables. https://home.engineering.iastate.edu/~alexs/classes/2021_Fall_281/labs/Lab_03/DE2-115_PIN_ASSIGNMENTS.pdf. Accessed: Feb. 2026.

[3] M. Morris Mano. *Computer System Architecture*. Prentice Hall, 3rd edition, 1992.

[4] University of Florida. Wire and bus connections in quartus. https://mil.ufl.edu/3701/docs/quartus/wire_bus.pdf. Accessed: Feb. 2026.